

Functions as Values

A clear and easy guide to using functions like variables in JavaScript

1. What Does 'Function as a Value' Mean?

In JavaScript, **functions are not just actions** — they are also **values**. This means you can store a function inside a variable, pass it to another function, or even return it from a function.

Think of a function like a **recipe card**. You can hand the recipe card to someone and say: *"Run this whenever you are ready."* That is exactly what we do when we pass a function as a value.

■ In many other languages, functions are only "actions" — you cannot pass them around. JavaScript treats them as first-class citizens, meaning they can go anywhere a value can go.

2. Regular Function Declaration

This is the most common way to write a function. You give it a name and define what it does.

Example from your code:

```
function handleTimeout() {  
  console.log('Timed out!');  
}
```

handleTimeout is the function name. The code inside the curly braces **{ }** runs when the function is called. Notice we can also use this function name as a value — we can give it to **setTimeout** without calling it yet (no parentheses).

3. Function Stored in a Variable (Arrow Function)

Instead of naming the function directly, we can store it inside a **const** variable. This is called a **function expression**.

Example from your code:

```
const handleTimeout2 = () => {  
  console.log('Timed out ... again!');  
};
```

Feature	Regular Function	Arrow Function
---------	------------------	----------------

Syntax	<code>function name() {}</code>	<code>const name = () => {}</code>
Keyword needed?	Yes — function	No — uses =>
Stored in variable?	Not directly	Yes, always
Common use	General purpose	Callbacks, short tasks

4. Passing a Function to setTimeout

setTimeout is a built-in JavaScript function. It takes two things:

1. A **function** to run later
2. A **number** (in milliseconds) — how long to wait

Examples from your code:

```
setTimeout(handleTimeout, 2000); // runs after 2 seconds
setTimeout(handleTimeout2, 3000); // runs after 3 seconds

setTimeout(() => {
  console.log('More timing out...');
}, 4000); // runs after 4 seconds
```

■ Notice: we write `handleTimeout` — NOT `handleTimeout()`. Writing `()` calls the function right now. Without `()`, we are just handing the function to `setTimeout`, which will call it later.

5. Passing a Function as an Argument (greeter example)

Your second image shows a function called **greeter** that accepts another function as its argument and then calls it.

Example from your code:

```
function greeter(greetFn) {
  greetFn(); // calls whatever function was passed in
}

greeter(() => console.log("Hi")); // passing an arrow function
```

Step	What happens
1	We call <code>greeter()</code> and pass an arrow function as the argument
2	Inside <code>greeter</code> , that arrow function is stored in the parameter <code>greetFn</code>

3	<code>greetFn()</code> is called — this runs the arrow function we passed
4	"Hi" is printed to the console

6. What is a Callback Function?

When you pass a function to another function so it can be called later, that passed function is called a **callback**.

Term	Meaning
Callback	A function passed into another function to be called later
Higher-order function	A function that accepts or returns another function

■ In your code: `handleTimeout` is the callback passed to `setTimeout`. `greetFn` is the callback passed to `greeter`.

7. Quick Summary

Concept	Example	Key Point
Named function	<pre>function handleTimeout() {}</pre>	Classic way to define a function
Arrow function in a variable	<pre>const fn = () => {}</pre>	Stored like any other value
Passing to <code>setTimeout</code>	<pre>setTimeout(fn, 2000)</pre>	No <code>()</code> — just hand the function
Inline arrow function	<pre>setTimeout(() => {...}, 4000)</pre>	Write the function directly inside
Function as argument	<pre>greeter(() => console.log("Hi"))</pre>	The callback pattern